



Computer Organization and Design

A Comprehensive CDA Course Review

University of Central Florida

Copyright © 2026 Student

UNIVERSITY OF CENTRAL FLORIDA

First printing, April 2026

Contents

I Foundations

1	Introduction to Computer Architecture	9
1.1	What is Computer Design and Architecture (CDA)?	9
1.1.1	Core Focus Areas	9
1.1.2	The Levels of Abstraction	10
1.2	Basic Computer Organization	10
1.3	The Eight Great Ideas in Computer Architecture	11
1.4	Moore's Law and Its Implications	11
1.4.1	Key Implications	11
1.5	Performance Metrics	12
1.5.1	Relative Performance	12
1.5.2	The CPU Time Equation	12
1.6	The Power Wall and Multicore Transition	13
1.7	Classes of Computing Applications	13
2	Number Systems and Binary Representation	15
2.1	Representations of Integers	15
2.2	Base Conversions	15
2.2.1	Binary to Decimal (Unsigned)	15
2.2.2	Decimal to Binary (Unsigned)	16

2.3	Signed Integer Representations	16
2.3.1	Method 1: Sign-Magnitude	17
2.3.2	Method 2: 1's Complement	17
2.3.3	Method 3: 2's Complement	17
2.3.4	Sign Extension	18
2.4	Binary Arithmetic and Overflow Detection	18
2.4.1	Binary Addition and Subtraction	18
2.4.2	Overflow Detection	18

II

Digital Logic Design

3	Combinational Logic	23
3.1	Logic Gates	23
3.1.1	Fundamental Gates	23
3.1.2	Universal and Advanced Gates	24
3.2	Boolean Algebra	25
3.2.1	Axioms and Theorems	25
3.2.2	Proving Equivalence	26
3.3	Canonical Forms and Truth Tables	26
3.3.1	Sum of Products (SOP)	26
3.3.2	Product of Sums (POS)	26
3.3.3	Incompletely Specified Functions	27
3.4	Karnaugh Maps (K-Maps) and Minimization	27
3.4.1	Terminology	27
3.4.2	Minimization Algorithm (MSP)	27
3.5	Standard Combinational Hardware	28
3.5.1	Adders	28
3.5.2	Decoders	28
3.5.3	Multiplexers (MUX)	29
3.5.4	Programmable Logic Arrays (PLA)	29
3.6	Logic Design Procedure	29
4	Sequential Logic Design	31
4.1	Introduction to Sequential Logic	31
4.2	Latches	31
4.2.1	Set-Reset (S-R) Latch	32
4.2.2	Clocked (Gated) S-R Latch	32
4.3	Flip-Flops	32
4.3.1	D Flip-Flop (Data Flip-Flop)	33
4.3.2	T (Toggle) and J-K Flip-Flops	33
4.4	Timing Methodologies	34
4.5	Counters & Registers	34
4.6	Finite State Machines (FSMs)	35
4.6.1	FSM Design Steps	35
4.6.2	Moore vs. Mealy Machines	35

III

Computer Arithmetic

5	Arithmetic Hardware and Algorithms	39
5.1	Integer Arithmetic	39
5.1.1	Addition and Subtraction	39
5.1.2	Integer Multiplication	41
5.1.3	Integer Division	41
5.2	Booth's Algorithm	42
5.3	IEEE 754 Floating-Point	43
5.3.1	Representation	43
5.3.2	Floating-Point Addition Algorithm	43
5.3.3	Floating-Point Multiplication Algorithm	43

IV

The MIPS Architecture

6	MIPS Instruction Set Architecture (ISA)	47
6.1	Register File	47
6.1.1	MIPS Register Conventions	48
6.1.2	Special-Purpose Registers	49
6.2	Memory Layout	49
6.2.1	Memory Segments	49
6.3	MIPS Instruction Formats	50
6.3.1	R-Type (Register Format)	50
6.3.2	I-Type (Immediate Format)	50
6.3.3	J-Type (Jump Format)	50
6.4	Addressing Modes	51
7	Assembly Programming	53
7.1	Introduction to MIPS Assembly	53
7.2	Conditionals and Branching	53
7.3	Loops (While, For, Do-While)	54
7.3.1	For Loops	55
7.3.2	Do-While Loops	55
7.4	Procedure Calls	55
7.5	Stack Usage	56
7.5.1	Mechanism (Prologue and Epilogue)	56
7.6	Recursion in MIPS	57

V

Processor Microarchitecture

8	The Datapath and Control	61
8.1	Single-Cycle Datapath Overview	61
8.1.1	Datapath Components	61

8.2	Control Signals	63
8.2.1	Signal Definitions and Actions	63
8.2.2	Control Signal Values by Instruction	63
8.3	ALU Control	64
8.3.1	Multilevel Decoding	64
8.3.2	ALU Control Logic Table	64
8.4	Data Flow by Instruction Type	64
8.4.1	R-Type Instructions (add, sub, and, or, slt)	65
8.4.2	Load Word (lw)	65
8.4.3	Store Word (sw)	65
8.4.4	Branch on Equal (beq)	65
8.4.5	Jump (j)	66
9	Exceptions and Interrupts	67
9.1	Introduction to Unexpected Events	67
9.2	Evolution of Exception Handling (VM/0 Conceptual Model)	68
9.3	MIPS Exception Handling Implementation	69
9.3.1	The Exception Program Counter (EPC)	69
9.3.2	The Cause and Status Registers	69
9.3.3	Handler Actions	70
9.4	Exceptions in a Pipelined Datapath	70
9.4.1	Datapath Modifications	70
9.4.2	Precise vs. Imprecise Exceptions	71

VI

Review and Projects

10	Midterm & Final Synthesis	75
10.1	Midterm Review & Core Concepts	75
10.1.1	Binary Conversions & Representations	75
10.1.2	Binary Arithmetic & ALU Operations	76
10.1.3	Combinational & Sequential Logic Design	77
10.2	Final Synthesis - MIPS Processor Simulation Project (spimcore)	77
10.2.1	Project Overview	77
10.2.2	Memory Architecture Details	78
10.3	Datapath Execution Stages (Implementation in C)	78
10.3.1	instruction_fetch	78
10.3.2	instruction_partition	78
10.3.3	instruction_decode	78
10.3.4	read_register	79
10.3.5	sign_extend	79
10.3.6	ALU_operations	79
10.3.7	rw_memory	79
10.3.8	write_register	79
10.3.9	PC_update	79
10.4	Critical Control Signals Reference	79



Foundations

1	Introduction to Computer Architecture .	9
1.1	What is Computer Design and Architecture (CDA)?	
1.2	Basic Computer Organization	
1.3	The Eight Great Ideas in Computer Architecture	
1.4	Moore's Law and Its Implications	
1.5	Performance Metrics	
1.6	The Power Wall and Multicore Transition	
1.7	Classes of Computing Applications	
2	Number Systems and Binary Representation	15
2.1	Representations of Integers	
2.2	Base Conversions	
2.3	Signed Integer Representations	
2.4	Binary Arithmetic and Overflow Detection	

1. Introduction to Computer Architecture

This chapter covers the fundamental concepts of Computer Design and Architecture (CDA) as presented in the textbook and the introductory course materials.

1.1 What is Computer Design and Architecture (CDA)?

Definition 1.1.1 — Computer Design and Architecture. Computer Design and Architecture (CDA) is the study of the structure, behavior, and implementation of computer systems. It focuses on the **Hardware/Software Interface**, bridging the gap between high-level programming languages and the physical hardware that executes them.

1.1.1 Core Focus Areas

According to the UCF CDA course introduction, the primary areas of focus include:

- **System Architecture:** The high-level structure and organization of a computer system.
- **Processor Design (Microarchitecture):** The internal organization of a processor, defining how it implements an Instruction Set Architecture (ISA).
- **Logic Design:** The design of circuits using logic gates to perform operations.
- **Assembly Language:** Low-level programming that provides a human-readable representation of machine instructions.
- **Machine Instructions:** The binary code (0s and 1s) that the hardware directly understands.

1.1.2 The Levels of Abstraction

Modern computer systems are designed using multiple levels of abstraction to manage complexity:

1. **Application Software:** Written in high-level languages such as C, Java, or Python.
2. **Systems Software:**
 - **Compiler:** Translates high-level code into assembly language.
 - **Operating System (OS):** Manages hardware resources and provides services to applications.
3. **Hardware:** The physical components (transistors, circuits) that execute instructions.

1.2 Basic Computer Organization

A computer system consists of five classic components that work together to process data:

Vocabulary 1.1 — Input. Devices that feed data into the computer (e.g., keyboard, mouse, touch screen).

Vocabulary 1.2 — Output. Devices that present data to the user (e.g., monitor, speakers, printer).

Vocabulary 1.3 — Memory. Stores data and instructions currently in use.

- **Volatile Memory (RAM):** Fast, but loses data when power is off.
- **Non-volatile Memory (SSD/HDD):** Slower, but retains data.

Vocabulary 1.4 — Datapath. The “brawn” of the processor; performs arithmetic and logical operations.

Vocabulary 1.5 — Control. The “brain” of the processor; commands the datapath, memory, and I/O devices according to the program instructions.

R The **Central Processing Unit (CPU)** or **Processor** is the combination of the **Datapath** and **Control** units.

1.3 The Eight Great Ideas in Computer Architecture

These foundational principles guide modern computer design:

- **Design for Moore’s Law:** Architects must anticipate where technology will be when the design finishes, rather than designing for today’s constraints.
- **Use Abstraction to Simplify Design:** Hiding low-level details allows designers to manage complexity and improve productivity.
- **Make the Common Case Fast:** Focus on optimizing the most frequent operations, which yields the greatest overall performance improvement.
- **Performance via Parallelism:** Executing multiple tasks simultaneously (e.g., multi-core processors).
- **Performance via Pipelining:** Overlapping the execution of multiple instructions (like an assembly line).
- **Performance via Prediction:** Guessing the outcome of an operation to start work early, rather than waiting (e.g., branch prediction).
- **Hierarchy of Memories:** Using a combination of small, fast, expensive memory (caches) and large, slow, cheap memory (DRAM/Disk) to provide speed and capacity.
- **Dependability via Redundancy:** Including extra components to detect and handle failures, making the system more reliable.

1.4 Moore’s Law and Its Implications

Definition 1.4.1 — Moore’s Law. Moore’s Law (proposed by Gordon Moore in 1965) predicted that the number of transistors per chip would double approximately every two years.

1.4.1 Key Implications

- **Exponential Growth:** Led to rapid improvements in performance and reductions in cost for decades.

- **The “Power Wall”:** As clock rates increased, power dissipation became a limiting factor, ending the era of rapid single-core frequency increases.
- **The Current State:** Moore’s Law is slowing down. Transistors are becoming harder and more expensive to shrink, leading to a shift toward **Domain-Specific Architectures (DSA)** and **Parallelism**.

1.5 Performance Metrics

Understanding performance requires distinguishing between response time and throughput.

Vocabulary 1.6 — Response Time. Also known as **Execution Time**, this is the total time required for a computer to complete a single task (user-centric).

Vocabulary 1.7 — Throughput. Also known as **Bandwidth**, this is the total amount of work done in a given time (system-centric).

1.5.1 Relative Performance

Proposition 1.5.1 — Performance Ratio. If Computer A is faster than Computer B, the relative performance is calculated as:

$$\text{Performance Ratio} = \frac{\text{Performance}_A}{\text{Performance}_B} = \frac{\text{Execution Time}_B}{\text{Execution Time}_A} \quad (1.1)$$

1.5.2 The CPU Time Equation

Theorem 1.5.2 — CPU Time Equation. The fundamental formula for calculating the CPU time of a program is:

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time} \quad (1.2)$$

Alternatively, using the clock rate:

$$\text{CPU Time} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}} \quad (1.3)$$

The components of this equation are determined by various factors:

- **Instruction Count:** Number of instructions executed (determined by the algorithm, compiler, and ISA).
- **CPI (Cycles Per Instruction):** The average number of clock cycles required per instruction (determined by the microarchitecture).
- **Clock Cycle Time:** The duration of one clock pulse (e.g., 0.25 ns for a 4 GHz clock).
- **Clock Rate:** The frequency of the clock (e.g., 4 GHz).

1.6 The Power Wall and Multicore Transition

For years, performance was improved by increasing clock frequency. However, power consumption increased proportionally to $\text{Voltage}^2 \times \text{Frequency}$.

Proposition 1.6.1 — Power Formula. The power consumption in CMOS technology is roughly:

$$\text{Power} = \frac{1}{2} \times \text{Capacitive Load} \times \text{Voltage}^2 \times \text{Frequency} \quad (1.4)$$

R Due to cooling limits (the “**Power Wall**”), architects could no longer increase frequency. This forced the industry to switch from **Uniprocessors** to **Multiprocessor** (Multicore) chips, where performance is gained through parallelism rather than raw speed.

1.7 Classes of Computing Applications

Modern computing is categorized into several distinct classes:

- **Personal Computers (PCs):** Focus on single-user performance and low cost.
- **Servers:** Focus on high capacity, reliability, and security; accessed over a network.
- **Supercomputers:** High-end scientific/engineering calculations; hundreds of thousands of processors.
- **Embedded Computers:** Computers hidden inside other devices (e.g., cars, appliances), often with strict power/cost constraints.
- **Personal Mobile Devices (PMDs):** Battery-powered, wireless devices like smartphones and tablets.
- **Cloud Computing / Warehouse Scale Computers (WSC):** Giant datacenters (like Google or Amazon) providing Software as a Service (SaaS).

2. Number Systems and Binary Representation

2.1 Representations of Integers

In the modern world, we use decimal (base 10) notation to represent integers. However, we can represent numbers using any base b , where b is a positive integer greater than 1. Computers represent integers and perform arithmetic using binary (base 2) expansions, which only use the digits 0 and 1.

Theorem 2.1.1 — Base b Expansion. Let b be a positive integer greater than 1. Then if n is a positive integer, it can be expressed uniquely in the form:

$$n = a_k b^k + a_{k-1} b^{k-1} + \cdots + a_1 b + a_0$$

where k is a nonnegative integer and $a_0, a_1, \dots, a_k$ are nonnegative integers less than b .

2.2 Base Conversions

2.2.1 Binary to Decimal (Unsigned)

To convert an unsigned binary number to decimal, express the binary number n in the polynomial form using base $b = 2$:

$$n = a_k 2^k + a_{k-1} 2^{k-1} + \cdots + a_1 2^1 + a_0 2^0$$

Start with $k = 0$ for the least significant bit (rightmost) and count upward to the most significant bit (leftmost).

■ **Example 2.1 — Binary to Decimal.** Convert $(11011)_2$ to decimal.

- $(1 \cdot 2^4) + (1 \cdot 2^3) + (0 \cdot 2^2) + (1 \cdot 2^1) + (1 \cdot 2^0)$
- $= 16 + 8 + 0 + 2 + 1 = 27$

2.2.2 Decimal to Binary (Unsigned)

To construct the base b expansion of an integer n (such as converting to binary where $b = 2$, or potentially to octal/hex by changing b):

1. Divide n by b using integer division to obtain a quotient and remainder. ($n = b \cdot q_0 + a_0$)
2. The remainder, a_0 , is the rightmost (least significant) digit.
3. Next, divide the quotient q_0 by b . ($q_0 = b \cdot q_1 + a_1$)
4. The new remainder, a_1 , is the next digit to the left.
5. Continue successively dividing the quotients by b until the quotient is 0. The remainders read from bottom to top (or last to first) form the binary result.

■ **Example 2.2 — Decimal to Binary.** Convert $(19)_{10}$ to binary.

- $19 \div 2 = 9 \text{ R } 1$ (Least Significant Bit)
- $9 \div 2 = 4 \text{ R } 1$
- $4 \div 2 = 2 \text{ R } 0$
- $2 \div 2 = 1 \text{ R } 0$
- $1 \div 2 = 0 \text{ R } 1$ (Most Significant Bit)

Result: $(10011)_2$

2.3 Signed Integer Representations

To represent negative numbers, we use a **sign bit**.

Vocabulary 2.1 — Sign Bit. The most significant bit (MSB) that acts as the sign: 0 represents a positive number, and 1 represents a negative number.

2.3.1 Method 1: Sign-Magnitude

The first bit represents the sign, and the remaining bits represent the absolute value (magnitude).

R It features cumbersome addition/subtraction (must compare magnitudes to determine the sign of the result) and has two representations for zero (+0 and −0).

■ **Example 2.3 — Sign-Magnitude.** Represent −55 in 8 bits. Magnitude of 55 = $(00110111)_2$. Set MSB to 1 → $(10110111)_2$.

2.3.2 Method 2: 1's Complement

If a number is negative, its representation is the bitwise inverse (complement) of its positive counterpart.

R Like Sign-Magnitude, it still has the "Zero Problem" with two representations of zero: +0 = 0000 and −0 = 1111.

■ **Example 2.4 — 1's Complement.** Represent −55 in 8 bits. Positive 55 = $(00110111)_2$. Bitwise inverse → $(11001000)_2$.

2.3.3 Method 3: 2's Complement

This is the standard representation for signed integers in modern computing (e.g., used by the `int` data type in C).

To represent a negative value:

1. Find the binary representation of the positive magnitude.
2. Perform a bitwise inverse (1's complement).
3. Add 1 to the result.

Proposition 2.3.1 — 2's Complement Advantage. It fixes the Zero Problem. $+0 = 0000$. Inverting yields 1111. Adding 1 yields 0000 (the carry-out is ignored), resulting in a single zero. It also simplifies binary arithmetic.

■ **Example 2.5 — 2's Complement.** Represent -55 in 8 bits. Positive $55 = (00110111)_2$. Inverse $\rightarrow 11001000$. Add 1 $\rightarrow (11001001)_2$.

2.3.4 Sign Extension

When moving a value to a larger data type (e.g., 4 bits to 8 bits), we use sign extension to fill the new bits:

- **Unsigned:** Pad with leading 0s (e.g., $0110 \rightarrow 0000\ 0110$).
- **Signed (2's Complement):** Pad with copies of the MSB.
 - Positive: $0110 \rightarrow 0000\ 0110$
 - Negative: $1010 \rightarrow 1111\ 1010$

2.4 Binary Arithmetic and Overflow Detection

2.4.1 Binary Addition and Subtraction

Binary numbers are added the same way decimal numbers are added. For subtraction, we implement it by adding the 2's complement equivalent of the subtrahend: $A - B = A + (-B)$. The extra carry-out from the MSB during 2's complement arithmetic is normally ignored.

2.4.2 Overflow Detection

Definition 2.4.1 — Overflow. Overflow occurs when the result of an arithmetic operation is too large or too small to be represented in the allocated number of bits.

Unsigned Addition:

In unsigned addition, an overflow occurs simply if the final carry-out from the most significant bit is 1.

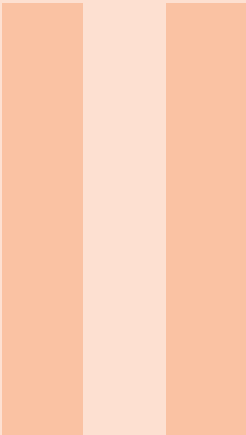
Signed Addition (2's Complement):

In signed addition, examining the final carry-out is not enough. Overflow happens when:

- Two positive numbers are added, and the result appears negative.
- Two negative numbers are added, and the result appears positive.
- *(Note: Adding a positive and a negative number can never cause an overflow).*

Theorem 2.4.1 — The XOR Rule for Signed Overflow. An easy hardware/algorithmic way to spot overflow is to compare the "Carry-in" of the MSB to the "Carry-out" of the MSB.

- If $\text{CarryIn}(\text{MSB}) \neq \text{CarryOut}(\text{MSB})$ (Mathematically: $\text{CarryIn} \oplus \text{CarryOut} = 1$), then an overflow has occurred.
- If they are the same, no overflow occurred.



Digital Logic Design

3	Combinational Logic	23
3.1	Logic Gates	
3.2	Boolean Algebra	
3.3	Canonical Forms and Truth Tables	
3.4	Karnaugh Maps (K-Maps) and Minimization	
3.5	Standard Combinational Hardware	
3.6	Logic Design Procedure	
4	Sequential Logic Design	31
4.1	Introduction to Sequential Logic	
4.2	Latches	
4.3	Flip-Flops	
4.4	Timing Methodologies	
4.5	Counters & Registers	
4.6	Finite State Machines (FSMs)	

3. Combinational Logic

3.1 Logic Gates

Definition 3.1.1 — Combinational Logic. Combinational logic circuits are digital circuits where the output is strictly a function of the present inputs. These circuits do not possess feedback loops or internal memory states. The primary voltage levels in digital systems are represented as Low (0) and High (1).

3.1.1 Fundamental Gates

Vocabulary 3.1 — NOT (Inverter). A unary operator that flips the logic level of the input.

$$Z = X' = \bar{X} \tag{3.1}$$

Vocabulary 3.2 — AND. A logical product operation. The output is 1 only if all inputs are 1.

$$Z = X \cdot Y \tag{3.2}$$

Vocabulary 3.3 — OR. A logical sum operation. The output is 1 if at least one input is 1.

$$Z = X + Y \quad (3.3)$$

R The truth tables for these fundamental gates define their behavior across all possible input combinations.

X	Y	NOT X	$X \cdot Y$ (AND)	$X + Y$ (OR)
0	0	1	0	0
0	1	1	0	1
1	0	0	0	1
1	1	0	1	1

3.1.2 Universal and Advanced Gates

Vocabulary 3.4 — NAND. NOT-AND operation. The output is 0 only if all inputs are 1. It is a universal gate.

$$Z = (X \cdot Y)' = \overline{X \cdot Y} \quad (3.4)$$

Vocabulary 3.5 — NOR. NOT-OR operation. The output is 1 only if all inputs are 0. It is also a universal gate.

$$Z = (X + Y)' = \overline{X + Y} \quad (3.5)$$

Vocabulary 3.6 — XOR (Exclusive-OR). Represents "inequality" or "difference". The output is 1 if the inputs are different.

$$Z = X \oplus Y = X'Y + XY' \quad (3.6)$$

Vocabulary 3.7 — XNOR (Exclusive-NOR). Represents "equality" or "coincidence". The output is 1 if the inputs are the same.

$$Z = \overline{X \oplus Y} = XY + X'Y' \quad (3.7)$$

3.2 Boolean Algebra

Definition 3.2.1 — Boolean Algebra. Boolean algebra provides the mathematical foundation for digital logic design, dealing with binary variables and logic operators (AND, OR, NOT).

3.2.1 Axioms and Theorems

Theorem 3.2.1 — Boolean Axioms and Theorems. The following properties hold for any Boolean variables X, Y, Z :

- **Identity:** $X + 0 = X$; $X \cdot 1 = X$
- **Null (Dominance):** $X + 1 = 1$; $X \cdot 0 = 0$
- **Idempotency:** $X + X = X$; $X \cdot X = X$
- **Involution:** $(X')' = X$
- **Inverse (Complement):** $X + X' = 1$; $X \cdot X' = 0$
- **Commutative:** $X + Y = Y + X$; $X \cdot Y = Y \cdot X$
- **Associative:** $(X + Y) + Z = X + (Y + Z)$; $(X \cdot Y) \cdot Z = X \cdot (Y \cdot Z)$
- **Distributive:** $X \cdot (Y + Z) = (X \cdot Y) + (X \cdot Z)$; $X + (Y \cdot Z) = (X + Y) \cdot (X + Z)$
- **Uniting Theorem:** $X \cdot Y + X \cdot Y' = X$; $(X + Y) \cdot (X + Y') = X$
- **Absorption:** $X + X \cdot Y = X$; $X \cdot (X + Y) = X$

Proposition 3.2.2 — DeMorgan's Theorems. The complement of a sum is the product of the complements, and the complement of a product is the sum of the complements:

$$\overline{X + Y} = \overline{X} \cdot \overline{Y} \quad (3.8)$$

$$\overline{X \cdot Y} = \overline{X} + \overline{Y} \quad (3.9)$$

3.2.2 Proving Equivalence

Equivalence between Boolean expressions can be established through:

1. **Perfect Induction:** Constructing a complete truth table to verify that all possible input combinations yield the same output.
2. **Algebraic Manipulation:** Applying Boolean axioms to transform one expression into the other.

3.3 Canonical Forms and Truth Tables

 A truth table is the unique algebraic signature of a Boolean function. A function with n inputs requires a table with 2^n rows to represent all possible states.

3.3.1 Sum of Products (SOP)

Definition 3.3.1 — Sum of Products / Minterm Expansion. Also known as **Disjunctive Normal Form (DNF)**. A **minterm** is a product term where every variable in the function appears exactly once (e.g., $m_3 = A'BC$ for a 3-variable function). It represents an input combination for which the function evaluates to 1. SOP is constructed by OR-ing all minterms that correspond to an output of 1. *Notation:* $F(A, B, C) = \Sigma m(1, 3, 5, 6, 7)$

3.3.2 Product of Sums (POS)

Definition 3.3.2 — Product of Sums / Maxterm Expansion. Also known as **Conjunctive Normal Form (CNF)**. A **maxterm** is a sum term where every variable appears exactly once. It represents an input combination for which the function evaluates to 0. Maxterms are the logical inverse of minterms. POS is constructed by AND-ing all maxterms that correspond to an output of 0. *Notation:* $F(A, B, C) = \Pi M(0, 2, 4)$

3.3.3 Incompletely Specified Functions

Vocabulary 3.8 — Don't Cares. In some digital systems, certain input combinations are guaranteed never to occur (e.g., invalid states in a BCD encoder). These are marked as 'X' or 'd' in truth tables. During minimization, these can be treated as either 0 or 1 to achieve the most efficient hardware realization.

3.4 Karnaugh Maps (K-Maps) and Minimization

Definition 3.4.1 — Karnaugh Map. A K-Map is a visual tool used to simplify Boolean expressions. It structures the truth table into a grid where cells are arranged in Gray-code order, ensuring that adjacent cells differ by only one variable. This facilitates the application of the Uniting Theorem ($XY + XY' = X$) through visual grouping.

3.4.1 Terminology

Vocabulary 3.9 — Implicant. A group of one or more adjacent 1s (and don't cares) that form a rectangle with side lengths that are powers of 2.

Vocabulary 3.10 — Prime Implicant (PI). An implicant that cannot be further expanded into a larger group.

Vocabulary 3.11 — Essential Prime Implicant (EPI). A prime implicant that covers at least one '1' that is not covered by any other prime implicant. EPIs are mandatory components of the minimal expression.

3.4.2 Minimization Algorithm (MSP)

■ **Example 3.1 — Algorithm for Minimal Sum of Products.** To derive the simplest two-level logic:

1. Group all 1s (and 'd's if helpful) into the largest possible prime implicants.
2. Identify all Essential Prime Implicants and include them in the final sum.
3. For any remaining 1s not covered by EPIs, select the minimal set of remaining PIs to complete the coverage.

Tip: To find the minimal POS, perform the same steps by grouping the 0s instead of the 1s.

3.5 Standard Combinational Hardware

3.5.1 Adders

Definition 3.5.1 — Half Adder. A circuit that adds two 1-bit inputs (A , B) and produces a Sum and a Carry-Out.

- $Sum = A \oplus B$
- $CarryOut = A \cdot B$

Definition 3.5.2 — Full Adder. A circuit that adds three 1-bit inputs (A , B , and a Carry-In C_{in}).

- $Sum = A \oplus B \oplus C_{in}$
- $C_{out} = AB + AC_{in} + BC_{in}$

3.5.2 Decoders

Definition 3.5.3 — Decoder. A functional block with n inputs and 2^n outputs. For any input combination, exactly one output is asserted. Decoders can be used to generate all minterms of a function.

3.5.3 Multiplexers (MUX)

Definition 3.5.4 — Multiplexer. A data selector that directs one of 2^n data inputs to a single output based on n selection lines.

3.5.4 Programmable Logic Arrays (PLA)

Definition 3.5.5 — PLA. A programmable device used to implement SOP logic. It features a programmable AND array to generate product terms, followed by a programmable OR array to sum them.

3.6 Logic Design Procedure



The professional design flow for combinational logic is a systematic process:

1. **Specification:** Define inputs, outputs, and functional requirements.
2. **Truth Table:** Map all input combinations to the desired output behavior.
3. **Logic Minimization:** Use K-Maps to find the Minimal Sum of Products (MSP) or Minimal Product of Sums (MPS).
4. **Implementation:** Draw the gate-level diagram, optimizing for transistor count or propagation delay through gate sharing.

4. Sequential Logic Design

4.1 Introduction to Sequential Logic

Unlike combinational circuits where outputs depend purely on current inputs, sequential circuits incorporate memory.

Definition 4.1.1 — Sequential Circuits. Circuits whose outputs depend on both current inputs and the history of past inputs (previous outputs). They are characterized by having an internal **State**.

- **Memory Elements:** Hardware components that store the current state.
- **Combinational Logic:** Computes the *next state* and *outputs* based on the input and current state.

R Sequential design relies on **Steady-State Abstraction**. Outputs are only observed after sufficient time has elapsed for the system to settle, and state changes are synchronized by a clock signal.

4.2 Latches

Definition 4.2.1 — Latch. A fundamental memory element where the output equals the stored state, and the state changes whenever appropriate inputs change (event-driven).

4.2.1 Set-Reset (S-R) Latch

Built from cross-coupled NOR or NAND gates, the S-R latch can force its output to 0 (reset) or 1 (set).

S	R	Q(t+1)	State
0	0	Q(t)	Hold
0	1	0	Reset
1	0	1	Set
1	1	-	Unstable / Not Allowed

Table 4.1: Truth Table for a NOR-based S-R Latch

Proposition 4.2.1 — S-R Latch Characteristic Equation. The next state of an S-R latch can be described by:

$$Q(t + \Delta) = S + R'Q(t) \quad (4.1)$$

R The $S = 1, R = 1$ state is unstable because it forces both Q and Q' to 0 in a NOR latch. Transitioning out of this state can lead to unpredictable oscillations.

4.2.2 Clocked (Gated) S-R Latch

To control *when* inputs are sampled, a clock signal is added. The inputs to the cross-coupled gates can only change when the clock is asserted (high). When the clock is 0, the latch is “closed” and holds its value regardless of S and R .

4.3 Flip-Flops

Definition 4.3.1 — Flip-Flop. A memory element built from clocked latches that is **edge-triggered**. Its internal state changes *only* on a specific clock edge (rising or falling).

4.3.1 D Flip-Flop (Data Flip-Flop)

On the active clock edge, the output Q becomes equal to the input D .

Proposition 4.3.1 — D Flip-Flop Behavior.

$$Q(t+1) = D \quad (4.2)$$

R Flip-flops are often constructed using a **master-slave** arrangement of latches to prevent unstable states from propagating and to ensure edge-triggered behavior.

4.3.2 T (Toggle) and J-K Flip-Flops

Vocabulary 4.1 — T Flip-Flop. A flip-flop that toggles its state if $T = 1$ and holds if $T = 0$.

Proposition 4.3.2 — T Flip-Flop Characteristic Equation.

$$Q(t+1) = T \oplus Q(t) \quad (4.3)$$

Vocabulary 4.2 — J-K Flip-Flop. A versatile flip-flop that solves the unstable state issue of the S-R latch by toggling when both inputs are high.

Proposition 4.3.3 — J-K Flip-Flop Characteristic Equation.

$$Q(t+1) = JQ'(t) + K'Q(t) \quad (4.4)$$

J	K	Q(t+1)	Operation
0	0	Q(t)	Hold
0	1	0	Reset
1	0	1	Set
1	1	Q'(t)	Toggle

Table 4.2: J-K Flip-Flop Truth Table

4.4 Timing Methodologies

Vocabulary 4.3 — Duty Cycle. The ratio of time the clock is “high” to the total clock period.

Vocabulary 4.4 — Setup Time (T_{su}). Minimum time inputs must be valid and steady **before** the clock edge.

Vocabulary 4.5 — Hold Time (T_h). Minimum time inputs must be valid and steady **after** the clock edge.

R If inputs transition during the setup/hold window, the flip-flop may enter a **metastable** state. A **synchronizer** (typically two D flip-flops in series) is used to mitigate this risk for asynchronous inputs.

Vocabulary 4.6 — Clock Skew. The difference in time between when two state elements see the same clock edge.

4.5 Counters & Registers

- **Data Registers:** Hold multi-bit binary values, typically with a *Write Enable* signal.
- **Shift Registers:** Move stored bits left or right per clock cycle. A **Universal Shift Register** supports parallel load and bidirectional shifting.

- **Register Files:** Sets of registers indexed by address, using decoders for writing and multiplexers for reading.
- **Counters:** Sequence through a fixed set of patterns. **Grey Code** counters are often used to prevent transient errors by changing only one bit per step.

4.6 Finite State Machines (FSMs)

Sequential circuits are mathematically modeled as FSMs, consisting of states, transitions, and outputs.

4.6.1 FSM Design Steps

■ **Example 4.1 — 1001 Overlapping Pattern Recognizer.** To design a pattern recognizer:

1. **State Diagram:** Define states representing the progress in seeing the sequence (e.g., “Saw 1”, “Saw 10”, etc.).
2. **State Encoding:** Assign binary values to each state. If there are N states, $\lceil \log_2 N \rceil$ flip-flops are required.
3. **Boolean Expressions:** Use Karnaugh Maps to derive logic for flip-flop inputs (Next State Logic) and outputs.
4. **Circuit Realization:** Implement using flip-flops and combinational gates.

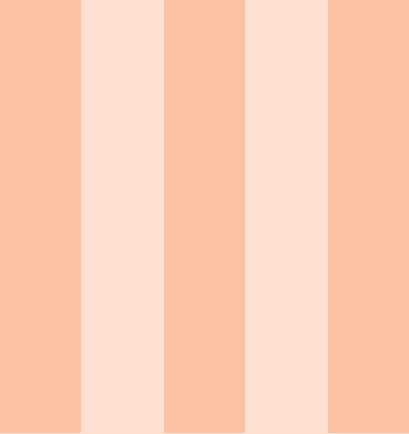
4.6.2 Moore vs. Mealy Machines

Definition 4.6.1 — Moore Machine. An FSM where the outputs depend **only** on the current state.

Definition 4.6.2 — Mealy Machine. An FSM where the outputs depend on **both** the current state and the current inputs.



Mealy machines can respond faster to input changes (within the same clock cycle) but may produce asynchronous glitches if the inputs are not synchronized.



Computer Arithmetic

5	Arithmetic Hardware and Algorithms	39
5.1	Integer Arithmetic	
5.2	Booth's Algorithm	
5.3	IEEE 754 Floating-Point	

5. Arithmetic Hardware and Algorithms

This chapter covers the foundational algorithms and hardware implementations for computer arithmetic, including integer addition, subtraction, multiplication, division, and IEEE 754 floating-point operations.

5.1 Integer Arithmetic

Vocabulary 5.1 — ALU. The **Arithmetic Logic Unit (ALU)** is the component of the processor responsible for performing arithmetic and logical operations.

5.1.1 Addition and Subtraction

Binary addition operates similarly to decimal addition.

Definition 5.1.1 — Half Adder. Adds two 1-bit inputs to produce a Sum and CarryOut.

- $\text{Sum} = A \oplus B$
- $\text{Cout} = A \cdot B$

Definition 5.1.2 — Full Adder. Adds two 1-bit inputs and a CarryIn to produce a Sum and CarryOut.

- $\text{Sum} = A \oplus B \oplus \text{Cin}$
- $\text{Cout} = A \cdot B + B \cdot \text{Cin} + A \cdot \text{Cin}$

Definition 5.1.3 — Ripple-Carry Adder. Cascades multiple 1-bit full adders to add multi-bit numbers. A significant disadvantage is the long propagation delay, as the carry bit may have to ripple from the least significant bit (LSB) to the most significant bit (MSB).

Definition 5.1.4 — Carry-Lookahead Adder. Solves the ripple delay by determining carry values sooner using Carry Generate ($G_i = A_i B_i$) and Carry Propagate ($P_i = A_i \oplus B_i$) logic.

R Subtraction ($A - B$) is performed using addition and two's complement representation: $A + (-B) = A + B' + 1$. The hardware takes the bitwise inverse of B (using a 2:1 multiplexer) and sets the initial CarryIn to 1.

Proposition 5.1.1 — Overflow Detection. Overflow occurs when the result is too large or too small to be represented in the available bits. In signed arithmetic, it can be detected by comparing the carry-in and carry-out of the most significant bit; if they are different, an overflow has occurred.

5.1.2 Integer Multiplication

Multiplication is conceptually based on the “paper and pencil” method of shifting and adding.

R [Evolution of Multipliers] Modern multiplication hardware has evolved through several versions to optimize space and speed:

- **Version 1:** Wastes space with a 64-bit Multiplicand register and a 64-bit ALU.
- **Version 2:** Reduces cost by shifting the Product right instead of the Multiplicand left.
- **Version 3 (Optimized):** Combines the Multiplier and Product registers into a single 64-bit register, using a 32-bit ALU.

■ **Example 5.1 — Multiplication by Constants.** Compilers often replace multiplications by constants with a series of shifts and adds to improve performance. For example, $A \times 4$ is equivalent to $A \ll 2$.

5.1.3 Integer Division

Binary division uses the same hardware components as multiplication but executes a different algorithm, typically **Restoring Division**.

Proposition 5.1.2 — Restoring Division Algorithm. The algorithm for X -bit division follows these steps for each pass:

1. Subtract the divisor from the left half of the remainder register.
2. Check the sign of the remainder:
 - **If negative ($\text{result} < 0$):** Restore the previous value by adding the divisor back. Shift the entire remainder register left, shifting in a 0 into the LSB.
 - **If positive ($\text{result} \geq 0$):** Do not restore. Shift the entire remainder register left, shifting in a 1 into the LSB.
3. After X iterations, shift the left half of the remainder register right by 1 bit.

The lower half of the register contains the **Quotient**, and the upper half contains the **Remainder**.

R In signed division, the sign of the remainder is always the same as the sign of the dividend.

5.2 Booth's Algorithm

Theorem 5.2.1 — Booth's Algorithm. Booth's Algorithm is an efficient method for multiplying two's complement signed integers. It allows multiplication without needing to strip signs and adjust them later. It is especially efficient for multipliers with long sequences (runs) of 1s or 0s.

Proposition 5.2.2 — Booth's Step-by-Step. Repeat X times for X -bit operands:

1. **Examine bits:** Look at the current LSB and the implied bit to its right:
 - 00: No arithmetic operation.
 - 01: Add Multiplicand to the left half of the Product.
 - 10: Subtract Multiplicand from the left half of the Product.
 - 11: No arithmetic operation.
2. **Arithmetic Shift Right (ASR):** Shift the entire Product register right by 1 bit, preserving the sign bit.

■ **Example 5.2 — Booth's Algorithm:** $(-5) \times 2$. Using 5-bit representation:

- Multiplier $(-5) = 11011$, Multiplicand $2 = 00010$.
- Initial Product: 00000 11011 0 (with implied bit 0).
- **Pass 1:** Ends are 10 $\rightarrow$ Subtract Multiplicand, then ASR.
- ... (Repeat for 5 total passes)
- Final 10-bit Product is obtained after dropping the implied bit.

5.3 IEEE 754 Floating-Point

The IEEE 754 standard defines the representation and arithmetic operations for real (“float”) numbers.

5.3.1 Representation

Definition 5.3.1 — IEEE 754 Fields.

- **Sign (1 bit):** 0 for positive, 1 for negative.
- **Exponent:** Stored as a biased exponent (Stored = Actual + Bias). 8 bits for Single Precision (Bias = 127), 11 bits for Double Precision (Bias = 1023).
- **Mantissa (Significand):** Represents the fractional part after the normalized leading 1 (which is implied and not stored).

Proposition 5.3.1 — Floating-Point Value Formula. The value of a normalized IEEE 754 number is given by:

$$(-1)^{\text{Sign}} \times (1.\text{Mantissa}) \times 2^{(\text{Exponent}-\text{Bias})} \quad (5.1)$$

5.3.2 Floating-Point Addition Algorithm

Proposition 5.3.2 — Addition Steps.

1. **Align Binary Points:** Denormalize the smaller value until its exponent matches the larger exponent.
2. **Add Significands:** Perform binary addition on the aligned significands.
3. **Normalize Result:** Shift significand and adjust exponent until format is 1.XXXX. Check for overflow/underflow.
4. **Round:** Round the significand. If rounding disrupts normalization, repeat Step 3.

5.3.3 Floating-Point Multiplication Algorithm

Proposition 5.3.3 — Multiplication Steps.

1. **Calculate Exponent:** New Biased Exponent = $(\text{Exp1} + \text{Exp2}) - \text{Bias}$.
2. **Multiply Significands:** Perform binary multiplication (including the implied leading 1).
3. **Normalize Product:** Shift right and increment exponent if necessary.
4. **Round:** Round the significand.
5. **Set Sign:** Positive if operands have same sign, negative otherwise.

IV

The MIPS Architecture

6	MIPS Instruction Set Architecture (ISA)	47
6.1	Register File	
6.2	Memory Layout	
6.3	MIPS Instruction Formats	
6.4	Addressing Modes	
7	Assembly Programming	53
7.1	Introduction to MIPS Assembly	
7.2	Conditionals and Branching	
7.3	Loops (While, For, Do-While)	
7.4	Procedure Calls	
7.5	Stack Usage	
7.6	Recursion in MIPS	

6. MIPS Instruction Set Architecture (ISA)

The MIPS (Microprocessor without Interlocked Pipelined Stages) architecture is a Reduced Instruction Set Computer (RISC) ISA that emphasizes simplicity, regularity, and efficiency. This chapter explores the fundamental components of the MIPS ISA, including its registers, memory organization, instruction formats, and addressing modes.

6.1 Register File

The MIPS architecture is built around a central **Register File** that acts as the primary hardware workspace. Instructions operate heavily on these registers rather than directly on memory.

Definition 6.1.1 — Register File. A **Register File** is a small, extremely fast set of storage locations within the CPU, typically constructed from flip-flops. MIPS provides 32 general-purpose registers, each 32 bits wide.

Vocabulary 6.1 — Word. In MIPS, a **word** is defined as a 32-bit (4-byte) entity.

Proposition 6.1.1 — Smaller is Faster. MIPS uses a limited set of 32 registers because smaller hardware structures are physically faster to access than larger ones (like main memory). This enables the high-speed execution characteristic of RISC architectures.

6.1.1 MIPS Register Conventions

While the hardware treats most registers identically, MIPS software follows a strict convention to ensure compatibility between different programs and libraries:

Name	Number	Usage	Preserved?
\$zero	0	Hardwired to 0	N/A
\$at	1	Assembler Temporary	No
\$v0-\$v1	2–3	Function results/expression evaluation	No
\$a0-\$a3	4–7	Function arguments	No
\$t0-\$t7	8–15	Temporaries (caller-saved)	No
\$s0-\$s7	16–23	Saved temporaries (callee-saved)	Yes
\$t8-\$t9	24–25	More temporaries	No
\$k0-\$k1	26–27	Reserved for OS Kernel	No
\$gp	28	Global Pointer	Yes
\$sp	29	Stack Pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return Address	Yes

Table 6.1: MIPS Register Convention

6.1.2 Special-Purpose Registers

- **Hi and Lo:** These 32-bit registers are used exclusively for multiplication and division.
 - *Multiply:* The 64-bit product is split; high 32 bits go to Hi, low 32 bits to Lo.
 - *Divide:* The quotient is stored in Lo, and the remainder is stored in Hi.
- **Floating-Point Registers:** A separate co-processor provides 32 floating-point registers (\$f0–\$f31) for single and double-precision operations.

6.2 Memory Layout

MIPS utilizes a structured memory layout to separate instructions, data, and execution state.

Definition 6.2.1 — Word Alignment. In a byte-addressed system like MIPS, **word alignment** requires that the starting memory address of a 32-bit word must be a multiple of 4 (e.g., 0, 4, 8, 12).

6.2.1 Memory Segments

- **Text Segment:** Stores the compiled machine code (instructions). Located at the lower end of memory.
- **Static Data:** Stores global variables and constants that exist for the program's lifetime.
- **Dynamic Data (Heap):** Area for runtime allocations (e.g., `malloc`). It grows *upward* from the static data area.
- **Stack:** Stores local variables and procedure call information. It starts at high memory and grows *downward* toward the heap.

■ **Example 6.1 — Stack Operations.** Since MIPS lacks explicit push and pop instructions, these are implemented using `addi`, `sw`, and `lw`:

```
# Push $t0 to stack
addi $sp, $sp, -4
sw $t0, 0($sp)

# Pop from stack to $t0
lw $t0, 0($sp)
addi $sp, $sp, 4
```

6.3 MIPS Instruction Formats

To maintain regularity, **every MIPS instruction is exactly 32 bits long**. There are three formats:

6.3.1 R-Type (Register Format)

Used for arithmetic and logical instructions where all operands are registers.

op (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
---------------	---------------	---------------	---------------	------------------	------------------

- **op:** Opcode (0 for R-type).
- **rs, rt:** Source registers.
- **rd:** Destination register.
- **shamt:** Shift amount (used for `sll`, `srl`).
- **funct:** Function code, identifying the specific operation.

6.3.2 I-Type (Immediate Format)

Used for instructions involving constants, memory access, or conditional branches.

op (6)	rs (5)	rt (5)	constant or address (16)
---------------	---------------	---------------	---------------------------------

- **op:** Unique opcode for the instruction.
- **rs:** Source/Base register.
- **rt:** Target register (destination for loads, source for stores/branches).
- **constant:** 16-bit signed immediate value or offset.

6.3.3 J-Type (Jump Format)

Used for unconditional jumps to distant memory locations.

op (6)	address (26)
--------	--------------

6.4 Addressing Modes

Addressing modes define how MIPS calculates the effective address of an operand or the target of a branch/jump.

1. **Register Addressing:** Operand is in a register (e.g., `add $t0, $t1, $t2`).
2. **Immediate Addressing:** Operand is a 16-bit constant in the instruction (e.g., `addi $t0, $t1, 100`).
3. **Base Addressing:** Address is Base Register + Offset (e.g., `lw $t0, 32($s3)`).
4. **PC-Relative Addressing:** Target is $(PC + 4) + (Offset * 4)$. Used for `beq` and `bne`.
5. **Pseudodirect Addressing:** Target is Upper 4 bits of PC : $(26\text{-bit Address} * 4)$. Used for `j` and `jal`.



In PC-Relative addressing, the offset is in *words*. The hardware automatically shifts the offset left by 2 bits to convert it to bytes before adding it to the PC.

7. Assembly Programming

7.1 Introduction to MIPS Assembly

When a higher-level programming language (like C) is compiled, it is first converted into an assembly language, and then processed into machine code (binary) capable of running on a processor. The “instruction set architecture” (ISA) represents the processor’s capability to run a particular set of commands. MIPS is a classic RISC (Reduced Instruction Set Computer) ISA.

Vocabulary 7.1 — MIPS Instructions. Individual commands that the MIPS processor can understand, categorized mainly into R-Type (Register), I-Type (Immediate), and J-Type (Jump).

7.2 Conditionals and Branching

Conditionals allow programs to execute different code paths based on dynamic evaluations. In MIPS, this is achieved through branch and jump instructions.

Definition 7.2.1 — Branch Instructions.

- `beq $s1, $s2, L` (Branch if equal): If `$s1 == $s2`, jump to label `L`.
- `bne $s1, $s2, L` (Branch if not equal): If `$s1 != $s2`, jump to label `L`.

Branches are **I-Type instructions**. The offset (16 bits) is used to calculate the “branch target address”.

Definition 7.2.2 — Jump Instructions.

- `j L` (Jump): Unconditionally jump to label `L`.

Jumps are **J-Type instructions**. The 26-bit address is a “pseudo-direct address”. It is shifted left by 2 (multiplied by 4) and concatenated with the top 4 bits of the current Program Counter (PC) to calculate the next instruction address.

R [spimcore Implementation Context] In the `spimcore` project (`project.c`), the datapath for conditionals is explicit. For a `beq` (opcode 4), the control signals are set as `Branch = 1` and `ALUOp = 1` (subtraction). The ALU subtracts the two registers, and if the result is 0, the Zero flag is set. The `PC_update` function checks `if (Branch && Zero)` to add the left-shifted extended offset to the PC. For a jump `j` (opcode 2), `Jump = 1`, and the PC directly inherits the left-shifted 26-bit `jsec` combined with the upper 4 bits of the PC.

7.3 Loops (While, For, Do-While)

High-level loop constructs are synthesized in MIPS using a combination of initialization, condition checking (via `slt`, `slti`), branching (`beq`, `bne`), and unconditional jumps (`j`).

■ **Example 7.1 — While Loop Execution.** Consider the C code:

```

1 i = 0;
2 while (i <= 20) {
3     A[i] = i;
4     i++;
5 }

```

MIPS Assembly Conversion:

```

1      addi $s1, $zero, 0      # Initialize i = 0 in $s1
2 loop: slti $t1, $s1, 21      # Check if i < 21 (i.e., i <=
      20). $t1 = 1 if true
3      beq $t1, $zero, end     # If $t1 == 0 (i > 20), exit
      loop
4
5      # Calculate address of A[i]
6      sll $t2, $s1, 2         # $t2 = i * 4 (byte offset)
7      add $t2, $t2, $s0       # $t2 = base address of A ($s0)
      + offset
8
9      sw $s1, 0($t2)          # Store i ($s1) into A[i]
10     addi $s1, $s1, 1         # i++
11     j loop                   # Jump back to start of loop
12 end: ...                    # Loop termination

```

7.3.1 For Loops

Similar to while loops, but the initialization happens once before the loop label, and the increment happens right before the jump back to the loop.

7.3.2 Do-While Loops

The condition check (`slti`, `beq`) is placed at the *end* of the loop, branching back to the start if the condition is true. This saves an unconditional jump instruction.

7.4 Procedure Calls

Programming languages separate individual tasks into functions or methods. In assembly, these are called “procedures”.

Vocabulary 7.2 — Procedure Registers.

- \$a0 – \$a3: Argument registers used to pass parameters to the procedure.
- \$v0 – \$v1: Return value registers used to pass results back to the caller.
- \$ra: Return address register (\$31), storing where the procedure should return.

Definition 7.4.1 — Procedure Instructions.

- **Jump and Link (jal):** e.g., `jal 10000`. Sets `$ra = PC + 4` (saving the return address) and sets `PC = 10000` (jumping to the procedure).
- **Jump Register (jr):** e.g., `jr $ra`. Sets `PC = $ra`, returning control to the calling function.

7.5 Stack Usage

When executing procedures, the processor has a limited number of registers. If a procedure needs to use registers that the caller also needs, or if the procedure calls another procedure (overwriting `$ra`), we must use memory to temporarily save these values.

Definition 7.5.1 — The Stack. A region of memory used for dynamic storage during procedure execution. It typically grows downwards in memory.

- `$sp`: The Stack Pointer register (\$29). It points to the top (lowest address) of the stack.

7.5.1 Mechanism (Prologue and Epilogue)

- **Prologue:** The start of a procedure. We move the stack pointer down to allocate space, then store (`sw`) registers. *Example:* `addi $sp, $sp, -8` (allocate 8 bytes for 2 words), followed by `sw $ra, 4($sp)` and `sw $a0, 0($sp)`.
- **Epilogue:** The end of a procedure. We load (`lw`) the saved values back into the registers, then move the stack pointer up to deallocate the space. *Example:* `lw $ra, 4($sp)`, `lw $a0, 0($sp)`, followed by `addi $sp, $sp, 8`.

7.6 Recursion in MIPS

Recursion occurs when a procedure calls itself. Because every call to the procedure overwrites the return address (\$ra) and potentially the arguments (\$a0), a recursive function **must** utilize the stack heavily to preserve state across recursive descents.

■ Example 7.2 — Recursive Sum. Convert the C function:

```

1 int recSum(int n) {
2     if (n <= 1) return n;
3     else return n + recSum(n-1);
4 }

```

MIPS Assembly Implementation:

```

1 recSum: slti $t0, $a0, 2          # Base case check: if n <= 1
2         beq  $t0, $zero, else     # If n > 1, jump to recursive
3                                     else' branch
4
5         # Base case: return n
6         add  $v0, $a0, $zero      # Place n ($a0) into return
7                                     register ($v0)
8         jr   $ra                 # Return to caller
9
10 else:   # Recursive case: preserve state on stack
11         addi $sp, $sp, -8        # Make room on stack for 2
12                                     words
13         sw   $ra, 4($sp)         # Preserve the return address
14         sw   $a0, 0($sp)         # Preserve the original
15                                     argument (n)
16
17         # Prepare for recursive call: recSum(n-1)
18         addi $a0, $a0, -1        # Argument becomes n - 1
19         jal  recSum              # Jump and link to recSum
20
21         # Return from recursion: restore state
22         lw   $a0, 0($sp)         # Restore original n
23         lw   $ra, 4($sp)         # Restore original return
24                                     address
25         addi $sp, $sp, 8        # Deallocate stack space
26
27         # Calculate: n + recSum(n-1)
28         add  $v0, $v0, $a0       # $v0 currently holds recSum(n
29                                     -1), add n
30         jr   $ra               # Return to caller

```

-  Notice that without saving `$ra` on the stack, the `jal recSum` instruction would overwrite the original return address, causing an infinite loop upon trying to return. Without saving `$a0`, the final addition `add $v0, $v0, $a0` would use 1 (the base case argument) instead of the caller's value of `n`.



Processor Microarchitecture

8	The Datapath and Control	61
8.1	Single-Cycle Datapath Overview	
8.2	Control Signals	
8.3	ALU Control	
8.4	Data Flow by Instruction Type	
9	Exceptions and Interrupts	67
9.1	Introduction to Unexpected Events	
9.2	Evolution of Exception Handling (VM/0 Conceptual Model)	
9.3	MIPS Exception Handling Implementation	
9.4	Exceptions in a Pipelined Datapath	

8. The Datapath and Control

This study guide provides a comprehensive summary of the MIPS single-cycle processor datapath and control, focusing on component interactions, control signals, ALU control logic, and the execution data flow for various instruction types.

8.1 Single-Cycle Datapath Overview

The datapath consists of the functional units and connections necessary to execute MIPS instructions.

Definition 8.1.1 — Single-Cycle Implementation. A **single-cycle implementation** is a design where every instruction begins execution on one clock edge and completes execution on the next clock edge. Consequently, the clock cycle time must be long enough to accommodate the slowest instruction, which is typically the Load Word (lw) instruction.

8.1.1 Datapath Components

The datapath is divided into state elements and combinational elements.

State Elements (Sequential Logic)

State elements are components that have internal storage and depend on the clock signal.

Vocabulary 8.1 — Program Counter (PC). A register that holds the address of the current instruction being executed.

Vocabulary 8.2 — Instruction Memory. Memory that stores the program code. It is addressed by the PC to fetch instructions.

Vocabulary 8.3 — Register File. A collection of 32 32-bit registers. It features two read ports for retrieving operands and one write port for storing results.

Vocabulary 8.4 — Data Memory. Memory used by load and store instructions to read from or write to system memory.

Combinational Elements

Combinational elements are components whose outputs depend solely on their current inputs.

Vocabulary 8.5 — ALU (Arithmetic Logic Unit). The core component that performs arithmetic (e.g., addition, subtraction) and logical (e.g., AND, OR) operations.

Vocabulary 8.6 — Adders. Functional units used exclusively for addition, such as incrementing the PC ($PC + 4$) or calculating branch target addresses.

Vocabulary 8.7 — Multiplexers (MUX). Data selectors that choose between multiple input sources based on a specific control signal.

Vocabulary 8.8 — Sign-Extension Unit. A unit that converts a 16-bit immediate value into a 32-bit value by replicating the most significant bit (sign bit).

Vocabulary 8.9 — Shift Left 2. A component that multiplies a value by 4, used specifically for calculating branch and jump targets in MIPS.

R Because a single-cycle datapath cannot reuse the same resource during a single instruction's execution, some elements must be duplicated (e.g., separate Instruction and Data Memories, separate ALU and PC adders).

8.2 Control Signals

Control signals coordinate the flow of data through the multiplexers and functional units.

Definition 8.2.1 — Main Control Unit. The unit responsible for generating control signals based strictly on the 6-bit **Opcode** field (bits [31:26] of the instruction).

8.2.1 Signal Definitions and Actions

The following table defines the primary control signals and their functions when deasserted (0) or asserted (1).

Signal	Purpose	Deasserted (0)	Asserted (1)
RegDst	Destination register selection.	From <i>rt</i> field (20:16).	From <i>rd</i> field (15:11).
RegWrite	Write enable for Register File.	No write occurs.	Register receives write data.
ALUSrc	Second ALU operand selection.	Read data 2.	Sign-extended immediate.
MemRead	Data Memory read enable.	No read occurs.	Memory content at address is output.
MemWrite	Data Memory write enable.	No write occurs.	Memory content at address is replaced.
MemtoReg	Register write data source.	ALU result.	Data Memory output.
Branch	Conditional branch indicator.	PC replaced by $PC + 4$.	If ALU Zero is 1, PC replaced by target.
Jump	Unconditional jump indicator.	Normal PC routing.	PC replaced by jump target.

Table 8.1: MIPS Control Signal Definitions

8.2.2 Control Signal Values by Instruction

The Main Control Unit sets these signals based on the opcode of each instruction.

Instr.	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
R-type	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

Table 8.2: Control Signal Truth Table (X denotes "Don't Care")

8.3 ALU Control

The ALU requires a 4-bit control input to determine the specific operation to perform.

Vocabulary 8.10 — ALU Control Lines. The 4-bit signals that drive the ALU:

- 0000: AND
- 0001: OR
- 0010: Add
- 0110: Subtract
- 0111: Set-on-less-than (slt)
- 1100: NOR

8.3.1 Multilevel Decoding

MIPS uses multilevel decoding to reduce complexity. The Main Control Unit outputs a 2-bit **ALUOp** signal, which the ALU Control Unit then combines with the 6-bit **funct** field to generate the final 4-bit ALU control lines.

- Proposition 8.3.1 — ALUOp Definitions.**
- 00: Memory References (lw/sw) → ALU performs **Add**.
 - 01: Branches (beq) → ALU performs **Subtract**.
 - 10: R-type instructions → ALU operation determined by **funct** field.

8.3.2 ALU Control Logic Table

8.4 Data Flow by Instruction Type

-  Every instruction execution begins with an **Instruction Fetch**. The PC is used to read the instruction from Instruction Memory, and an adder calculates $PC + 4$.

ALUOp	Instr. Type	Funct Field	Action	Control Lines
00	lw / sw	XXXXXX	Add	0010
01	beq	XXXXXX	Subtract	0110
10	R-type (add)	100000	Add	0010
10	R-type (sub)	100010	Subtract	0110
10	R-type (and)	100100	AND	0000
10	R-type (or)	100101	OR	0001
10	R-type (slt)	101010	Set-on-less-than	0111

Table 8.3: ALU Control Logic Truth Table

8.4.1 R-Type Instructions (add, sub, and, or, slt)

1. **Read:** Bits [25:21] (rs) and [20:16] (rt) are sent to the Register File to read two source registers.
2. **Execute:** The ALUSrc MUX selects read data 2. The ALU performs the operation specified by the ALU Control unit.
3. **Write Back:** The RegDst MUX selects bits [15:11] (rd) as the write register. The MemtoReg MUX selects the ALU result. RegWrite is asserted.

8.4.2 Load Word (lw)

1. **Read:** Bits [25:21] (rs) read the base address register. The 16-bit immediate field (bits [15:0]) is sign-extended.
2. **Execute:** The ALUSrc MUX selects the sign-extended immediate. The ALU adds the base register and immediate to calculate the memory address.
3. **Memory Access:** MemRead is asserted. Data Memory reads the value at the address.
4. **Write Back:** The RegDst MUX selects bits [20:16] (rt) as the destination register. RegWrite is asserted.

8.4.3 Store Word (sw)

1. **Read:** rs reads the base address register; rt reads the data to be stored. The immediate is sign-extended.
2. **Execute:** The ALU adds the base register and the sign-extended immediate to calculate the address.
3. **Memory Access:** MemWrite is asserted. The value from rt is written into Data Memory.

8.4.4 Branch on Equal (beq)

1. **Read:** rs and rt are read. The 16-bit immediate is sign-extended and shifted left by 2 to form the offset.
2. **Target Calculation:** A dedicated adder adds the shifted offset to $PC + 4$.
3. **Comparison:** The ALU subtracts the two register values.
4. **Update PC:** If registers are equal (Zero signal is 1), the Branch Target Address is routed to the PC.

8.4.5 Jump (j)

1. **Fetch:** Instruction is fetched and $PC + 4$ is calculated.
2. **Address Calculation:** The 26-bit target address (bits [25:0]) is shifted left by 2.
3. **Update PC:** The 28-bit value is concatenated with the upper 4 bits of $PC + 4$. The Jump signal routes this absolute address to the PC.

9. Exceptions and Interrupts

9.1 Introduction to Unexpected Events

Handling unexpected events is a fundamental requirement for robust processor design. These events require a sudden change in the flow of control, pausing the current program to handle the underlying issue.

Definition 9.1.1 — Exception. An unexpected event that arises *within* the CPU during instruction execution. Examples include an undefined opcode, an arithmetic overflow, or a system call (syscall).

Definition 9.1.2 — Interrupt. An unexpected event originating from an *external* source, typically an I/O controller (e.g., a disk drive completing a transfer, or a hardware timer).

9.2 Evolution of Exception Handling (VM/0 Conceptual Model)

To understand how processors handle exceptions, we observe the conceptual addition of hardware to a simple sequential processor (VM/0).

Vocabulary 9.1 — Overflow (OV) Flip-Flop. A hardware flag set when an arithmetic operation results in an overflow.

Vocabulary 9.2 — NEWPC Register. A register containing the address of the error handler. If the OV flag is set, the CPU bypasses the normal flow and executes the instruction at this address.

Vocabulary 9.3 — FENCE Register. A register used for memory protection that stores the maximum memory boundary for a process.

Vocabulary 9.4 — MP (Memory Protection) Flip-Flop. A hardware flag that triggers if a memory request exceeds the bound set in the FENCE register.

Vocabulary 9.5 — Mode Bit. A bit indicating the current execution state (e.g., 0 for User, 1 for Privileged). Certain opcodes are restricted to Privileged mode.

Vocabulary 9.6 — PI (Privileged Instruction) Flag. A flag set if an unprivileged process attempts to execute restricted instructions.

Vocabulary 9.7 — OLDPC / OLDRO. Registers used to save the current Program Counter and data register (R0) respectively, allowing the CPU to pause a program for an I/O switch and later restore its state.

Definition 9.2.1 — Interrupt Handler Vector (IHV). A unified structure used to manage various exceptions. It is a table in system memory containing the starting addresses of specific handler routines, such as:

- Overflow Handler
- Timer Handler
- I/O Handler
- System Call Handler

9.3 MIPS Exception Handling Implementation

In the MIPS architecture, exceptions and interrupts are managed by the **System Control Coprocessor (CP0)**. When an exception occurs, the processor must freeze the execution state and identify the cause.

9.3.1 The Exception Program Counter (EPC)

Definition 9.3.1 — Exception Program Counter (EPC). A special 32-bit register within CP0 used to precisely save the address of the offending (or interrupted) instruction.

- **Restartable Exception:** If the exception is restartable (e.g., an I/O interrupt or an OS page fault), the handler takes corrective action and uses the EPC to return execution to the exact point the program was interrupted.
- **Fatal Exception:** If the exception is fatal (e.g., an undefined opcode or memory protection violation), the program is terminated, and the OS uses the EPC to report the exact location of the error.

9.3.2 The Cause and Status Registers

To determine why the exception occurred, MIPS utilizes specific registers in CP0:

Definition 9.3.2 — Cause Register. Saves an encoded indication of the exact problem. The handler reads this register to determine which specific corrective routine to execute.

Definition 9.3.3 — Status Register. Contains the operating mode, interrupt enabling flags, and diagnostic states. Specific codes indicate the exception type (e.g., 0000 for undefined instruction, 0180 for arithmetic overflow).

9.3.3 Handler Actions

When an exception is triggered, the processor automatically performs a sequence of events:

1. Reads the **Cause** register to identify the source.
2. Transfers control to the relevant handler routine (often utilizing the **IHV**).
3. The handler determines the required action.
4. If restartable, it corrects the issue and returns via the **EPC**. Otherwise, it terminates the process and issues an error report using the **EPC**.

9.4 Exceptions in a Pipelined Datapath

...

9.4.1 Datapath Modifications

R Consider an arithmetic overflow occurring during an add instruction that is currently in the Execution (EX) stage. The following steps must be taken:

1. **Prevent State Corruption:** Immediately prevent the offending instruction from overwriting registers or memory by disabling **RegWrite** and **MemWrite**.
2. **Complete Previous Instructions:** Instructions in the MEM and WB stages are allowed to safely complete.
3. **Flush the Pipeline:** The offending instruction and all subsequent instructions in the IF and ID stages are flushed.
4. **Record State:** Hardware sets the **Cause** and **EPC** register values.
5. **Transfer Control:** The PC is updated to the address of the exception handler.

9.4.2 Precise vs. Imprecise Exceptions

Because pipelining overlaps the execution of multiple instructions, it is possible for multiple exceptions to occur within the same clock cycle.

Definition 9.4.1 — Precise Exceptions. A mechanism where the processor handles the exception originating from the *earliest* instruction in the logical program order (the instruction furthest along in the pipeline).

Proposition 9.4.1 Any subsequent instructions in the pipeline, even if they also generated exceptions, are simply flushed. This ensures the processor state remains perfectly consistent, allowing the program to be accurately restarted or debugged sequentially.



Review and Projects

10	Midterm & Final Synthesis	75
10.1	Midterm Review & Core Concepts	
10.2	Final Synthesis - MIPS Processor Simulation Project (spimcore)	
10.3	Datapath Execution Stages (Implementation in C)	
10.4	Critical Control Signals Reference	

10. Midterm & Final Synthesis

This chapter provides a comprehensive review of key Computer Organization and Design (CDA) concepts, ranging from low-level binary conversions and combinational logic design to the complete MIPS processor datapath. It also serves as the ultimate synthesis by detailing the requirements and implementation of a MIPS processor simulator in C (the spimcore project).

10.1 Midterm Review & Core Concepts

10.1.1 Binary Conversions & Representations

R [Recall: Unsigned Integers] Unsigned integers are represented using the standard positional weighted sum, where each bit position i corresponds to a value of 2^i .

■ **Example 10.1 — Binary to Decimal Conversion.** Convert $(1100\ 1111)_2$ to decimal:

$$128 + 64 + 8 + 4 + 2 + 1 = 207_{10} \quad (10.1)$$

■ **Example 10.2 — Decimal to Binary (Unsigned).** To convert 55_{10} to binary, use successive division by 2, where remainders form the binary digits (least significant bit first):

$$55/2 = 27 \text{ remainder } 1$$

$$27/2 = 13 \text{ remainder } 1$$

$$13/2 = 6 \text{ remainder } 1$$

$$6/2 = 3 \text{ remainder } 0$$

$$3/2 = 1 \text{ remainder } 1$$

$$1/2 = 0 \text{ remainder } 1$$

Result: $(0011\ 0111)_2$

R [Recall: 2's Complement] 2's Complement is used to represent signed integers. The most significant bit (MSB) carries a negative weight. It is the standard representation for signed values in modern computer systems.

■ **Example 10.3 — Signed Conversions.**

- **Binary to Decimal:** $(1100\ 1111)_2$ has a sign bit of 1 (negative). Inverting the bits gives $(0011\ 0000)_2$, and adding 1 gives $(0011\ 0001)_2 = 49_{10}$. Therefore, the original value is -49_{10} .
- **Decimal to Binary:** To represent -55_{10} , first convert $+55$ to binary $(0011\ 0111)_2$, then invert the bits to $(1100\ 1000)_2$ and add 1 to get $(1100\ 1001)_2$.

10.1.2 Binary Arithmetic & ALU Operations

R [Recall: Binary Subtraction] $A - B$ is calculated using addition and the 2's complement of B :

$$A - B = A + (\text{bitwise inverse of } B) + 1 \quad (10.2)$$

R [Recall: Overflow Detection] Overflow occurs when the addition of two numbers with the same sign produces a result with the opposite sign. For example, adding two positive numbers yields a negative result, or adding two negative numbers yields a positive result.

R [Recall: Booth's Algorithm] Booth's Algorithm is a multiplication algorithm for signed 2's complement numbers that utilizes arithmetic shifting and conditional addition/subtraction.

1. Initialize the Product and set the Previous LSB to 0.
2. Analyze the current LSB and the Previous LSB:
 - 01: Add Multiplicand.
 - 10: Subtract Multiplicand.
 - 00 or 11: No arithmetic action.
3. Perform an Arithmetic Right Shift.

Vocabulary 10.1 — 32-bit ALU Interface. The Arithmetic Logic Unit (ALU) is the core of the processor's datapath. In the `spimcore` project, this is implemented in the `ALU` and `ALU_operations` functions.

- **Inputs:** A (32 bits), B (32 bits), `ALUControl` (control signal).
- **Outputs:** `ALUresult` (32 bits), Zero flag (1 bit).

10.1.3 Combinational & Sequential Logic Design

R [Recall: Karnaugh Maps (K-Maps)] K-Maps are used for boolean logic simplification to find the minimal sum-of-products equation. Groupings of adjacent 1s must be in powers of 2 (1, 2, 4, 8) and can wrap around the edges.

R [The Logic Design Process] The Logic Design Process involves defining inputs/outputs, creating a truth table, mapping to a K-Map, deriving a boolean equation, and drawing the logic gate diagram.

10.2 Final Synthesis - MIPS Processor Simulation Project (spimcore)

10.2.1 Project Overview

R The capstone project requires simulating the datapath and control of a complete MIPS processor using the C programming language. The core implementation resides in `project.c`, driven by the `spimcore.c` framework.

- Do **NOT** modify `spimcore.c` or `spimcore.h`.
- Compilation: `gcc -o spimcore spimcore.c project.c`.
- Execution: The program executes `.asc` files containing hexadecimal MIPS instructions.

10.2.2 Memory Architecture Details

Vocabulary 10.2 — Word-Aligned Array. The main memory (Mem) is modeled as an array of 32-bit unsigned integers (`unsigned int`).

Proposition 10.2.1 — Addressing & Indexing. Standard MIPS memory addresses are byte-addressed. Because the C array holds words (4 bytes), you must convert the byte address to a word index by shifting right by 2.

$$\text{Index} = \text{Address} \gg 2 \quad (10.3)$$

Example: `Mem[PC  $\gg$  2]`

R Address alignment must be tested *before* any shifting occurs. Functions like `rw_memory` and `instruction_fetch` must verify word alignment (`address % 4 == 0`) and halt if unaligned.

10.3 Datapath Execution Stages (Implementation in C)

The step-by-step execution of a MIPS instruction is divided into specific functions within `project.c`.

10.3.1 `instruction_fetch`

Retrieves the instruction from memory.

- Check alignment: `PC % 4 == 0`.
- Fetch: `Instruction = Mem[PC  $\gg$  2]`.

10.3.2 `instruction_partition`

Slices the 32-bit instruction into its component fields.

- Use bitwise AND (&) for masking and right shifts ($\gg$) for positioning.
- Example: `unsigned op = (instruction & 0xFC000000)  $\gg$  26;`
- Extract `r1`, `r2`, `r3`, `funct`, `offset`, and `jsec`.

10.3.3 `instruction_decode`

Sets the specific processor control signals in the `struct_controls` struct based on the op code.

- Signals are char types (0 or 1), except `ALUOp` (3 bits).

- For R-type instructions, `ALUOp` is set to 7 (111), signaling `ALU_operations` to decode the `funct` field.

10.3.4 `read_register`

Retrieves operand data from the Register File (Reg array) using `r1` and `r2`.

10.3.5 `sign_extend`

Extends a 16-bit immediate offset to 32 bits. If the sign bit (bit 15) is 1, the upper 16 bits must be padded with 1s.

10.3.6 `ALU_operations`

Prepares inputs and multiplexer logic for the ALU.

- Use `ALUSrc` to choose between `data2` and `extended_value`.
- If `ALUOp == 7`, decode `funct` to determine the specific operation.

10.3.7 `rw_memory`

Handles Data Memory operations.

- Verify alignment: `ALUresult % 4 == 0`.
- Read/Write using the index: `Mem[ALUresult >> 2]`.

10.3.8 `write_register`

Writes data back to the register file.

- Use `MemtoReg` to select data source (ALU or Memory).
- Use `RegDst` to select destination index (`r2` or `r3`).
- Commit change if `RegWrite == 1`.

10.3.9 `PC_update`

Updates the Program Counter for the next cycle.

- Sequential: `PC = PC + 4`.
- Branching: If `Branch == 1` and `Zero == 1`, add shifted offset to PC.
- Jumping: If `Jump == 1`, combine shifted `jsec` with upper bits of updated PC.

10.4 Critical Control Signals Reference

The following signals are defined in `spimcore.h` within the `struct_controls` structure.

Vocabulary 10.3 — `RegDst`. Determines the destination register index source. 0 for I-type (uses `r2`), 1 for R-type (uses `r3`).

Vocabulary 10.4 — ALUSrc. Determines the second input to the ALU. 0 for R-type and branches, 1 for most I-types.

Vocabulary 10.5 — MemtoReg. Selects the data to be written to the register file. 0 routes the ALU result, 1 routes Memory read data.

Vocabulary 10.6 — Zero. ALU output flag indicating a zero result, used with the Branch signal to resolve conditional branches.